电子科技大学
格拉斯哥学院
Glasgow College, UESTC

UESTC 3010: Team Design Project and Skills

Lecture 3b

Software Construction and Testing

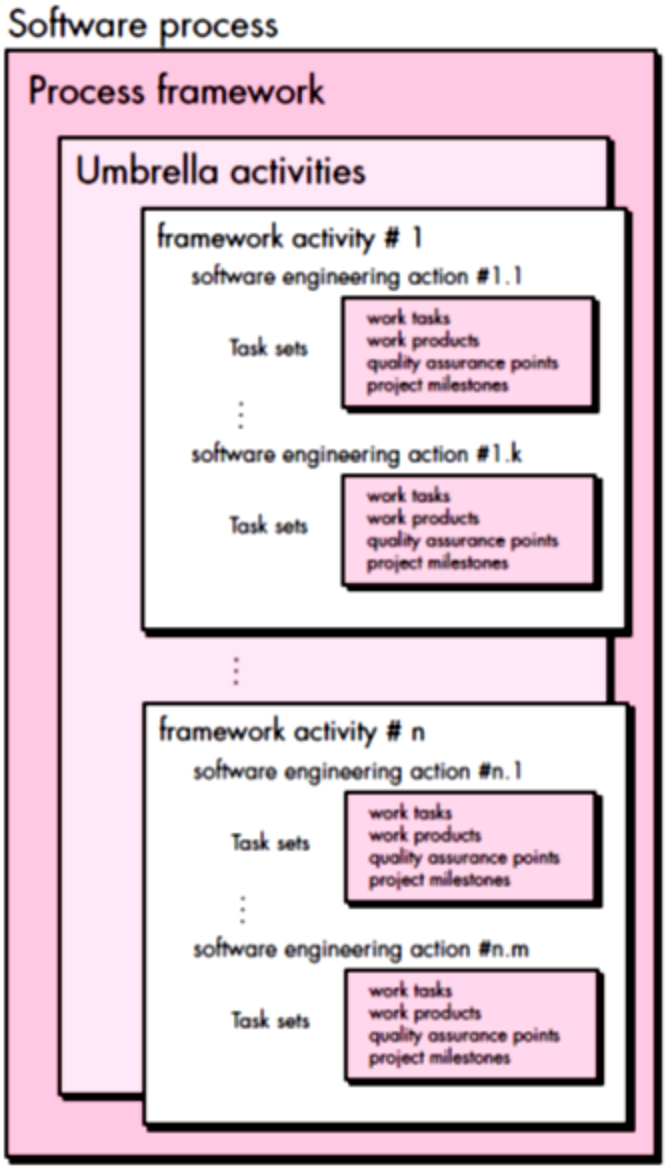
Software Construction Methodologies

Process Model

- A process is a collection of work activities, actions, and tasks that are performed when some work product is to be created
- Each of these activities, actions, and tasks reside within a framework or model that defines their relationship with the process and with one another
- A generic process framework for software engineering defines five framework activities
  - communication, planning, modelling, construction, and deployment
- In addition, a set of umbrella activities are applied throughout the process
  - project tracking and control, risk management, quality assurance, configuration management, technical reviews

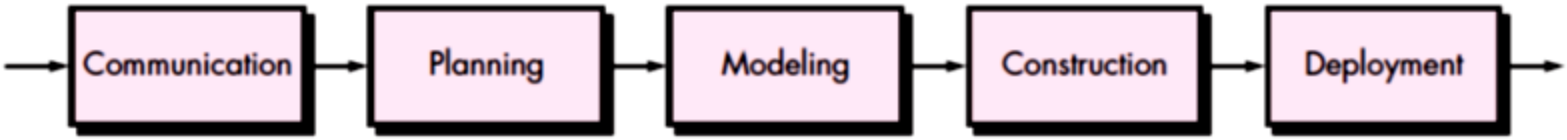
A Generic Process Model

- The software process is represented schematically in Figure
- Each framework activity is populated by a set of software engineering actions
- Each software engineering action is defined by a task set that identifies
  - the work tasks that are to be completed,
  - the work products that will be produced,
  - the quality assurance points that will be required, and
  - the milestones that will be used to indicate progress

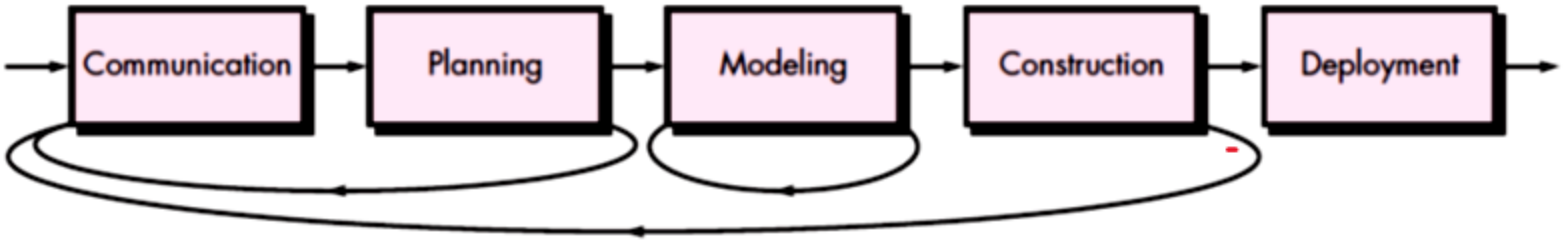


Process Flow (1)

- Process flow describes how the framework activities and the actions and tasks that occur within each framework activity are organized with respect to sequence and time
- A linear process flow executes each of the five framework activities in sequence, beginning with communication and culminating with deployment
- An iterative process flow repeats one or more of the activities before proceeding to the next



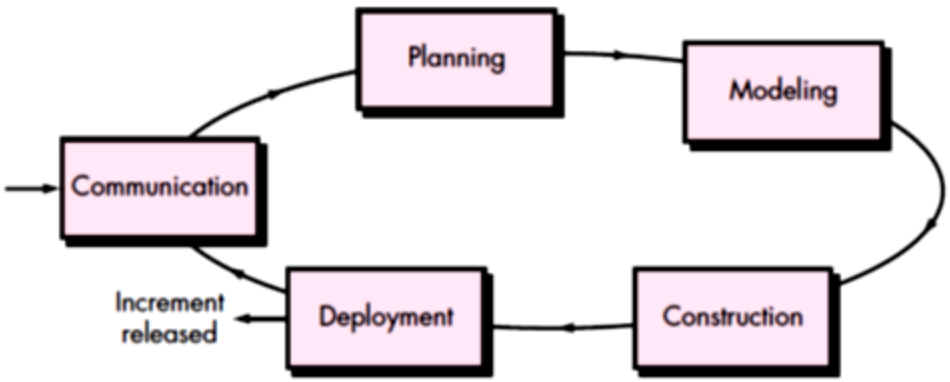
(a) Linear process flow



(b) Iterative process flow

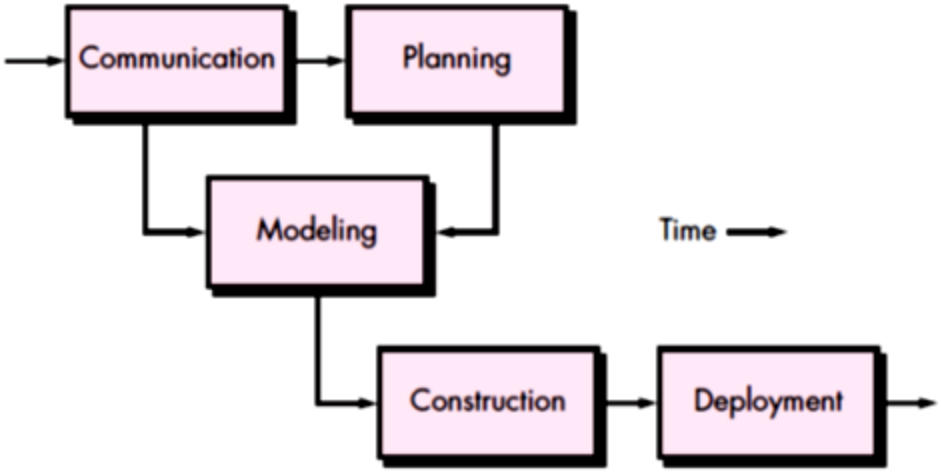
Process Flow (2)

- An evolutionary process flow executes the activities in a "circular" manner. Each circuit through the five activities leads to a more complete version of the software



(c) Evolutionary process flow

- A parallel process flow executes one or more activities in parallel with other activities (e.g., modelling for one aspect of the software might be executed in parallel with construction of another aspect of the software)



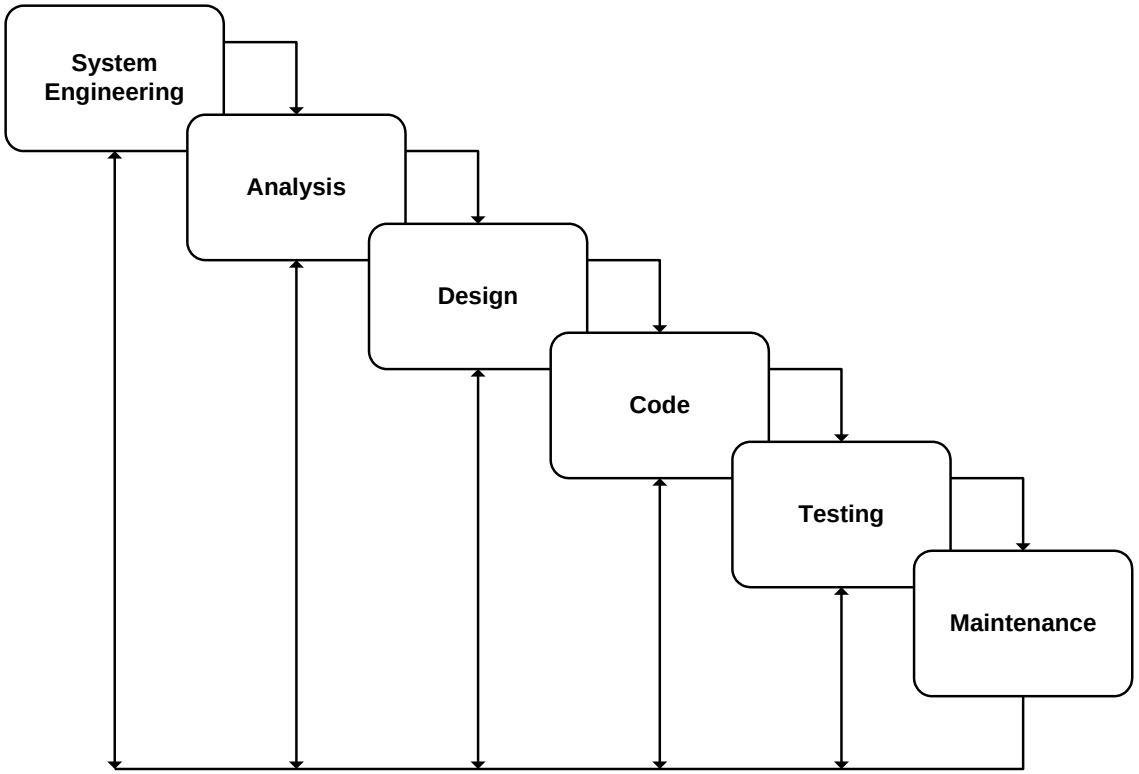
(d) Parallel process flow

Software Process Models

- We will look at three different methodologies:
  - The Classic Life Cycle/The Waterfall Model
  - Prototyping
  - The Spiral Model

The Classic Life Cycle/The Waterfall Model (1)

- suggests a systematic, sequential approach to software development that begins with customer specification of requirements and progresses through planning, modelling, construction, and deployment, culminating in ongoing support of the completed software

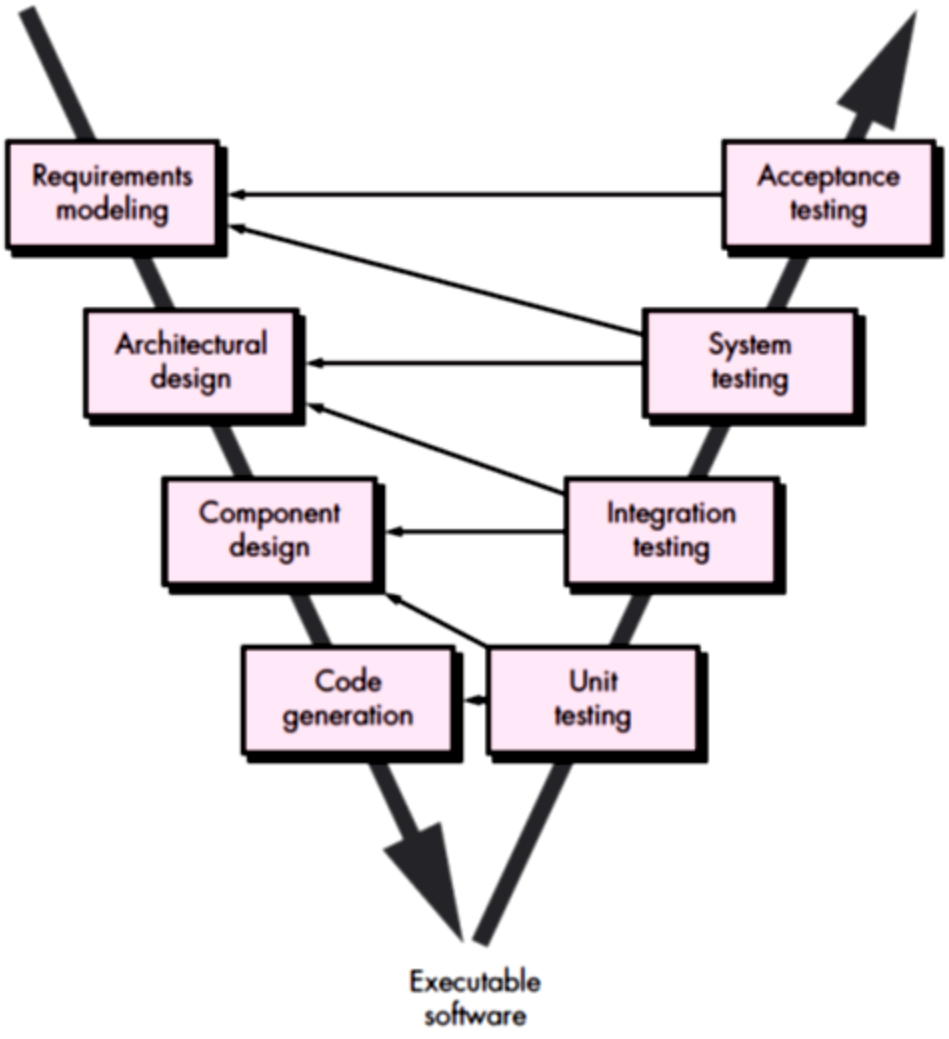


The Classic Life Cycle/The Waterfall Model (2)

- System Engineering: from the whole system which parts is the software going to achieve
- Analysis: this is a requirements spec for the software
- Design: look at the software in terms of data structures and processes to be performed upon the data
- Code: write the code!
- Testing: ensure the code behaves according to the spec
- Maintenance: perform alterations as required to correct faults

V-Model

- The V-model depicts the relationship of quality assurance actions to the actions associated with communication, modelling, and early construction activities



Problems with The Classic Life Cycle and V-Model

- Real projects rarely follow the sequential flow that is proposed by this methodology. Iteration occurs that creates problems in the application of the methodology
- The software is only available to the customer towards the end of the project. This means that errors made, if undetected until the customer review, can be disastrous
- An explicit set of requirements are required. These are often difficult for the customer to state as often uncertainty occurs at the start of many projects

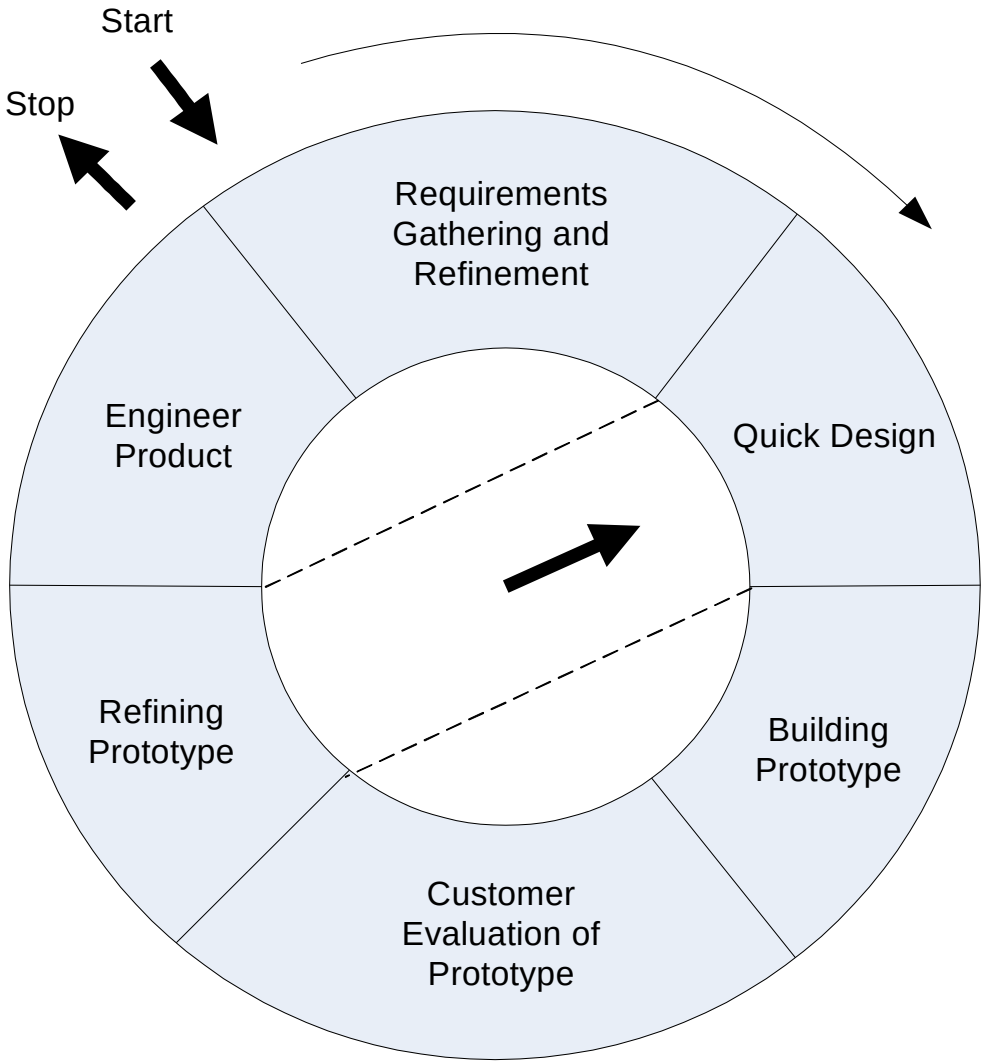
Evolutionary Process Models

- Software, like all complex systems, evolves over a period of time
- Business and product requirements often change as development proceeds, making a straight line path to an end product unrealistic
- Tight market deadlines make completion of a comprehensive software product impossible, but a limited version must be introduced to meet competitive or business pressure;
- A set of core product or system requirements is well understood, but the details of product or system extensions have yet to be defined
- In these and similar situations, you need a process model that has been explicitly designed to accommodate a product that evolves over time
- Evolutionary models are iterative. They are characterized in a manner that enables you to develop increasingly more complete versions of the software

Prototyping (1)

- Often the customer has defined a general set of objectives the software must meet, but has omitted detailed input, processing and/or output requirements
- Equally the developer may be unsure of the efficiency of an algorithm, operating system or the form a human-machine interface should take
- Prototyping is a methodology that helps to deal with these and similar issues

Prototyping (2)



Prototyping (3)

- Requirements gathering
  - Developer and customer meet and define overall objectives
  - Outline where further definition is mandatory
- Quick design
  - techniques leading to the production of a prototype (whole or partial)
- Building prototype
  - Prototype construction phase
- Customer evaluation
  - Prototype is evaluated by customer/user
  - Outcomes feed into refining prototype
- Refining Prototype
  - This is where the prototype begins to be 'tuned', as an iterative process leading back to the Quick design phase, to meet the customers needs
  - Also enables the developer to better understand what needs to be done

Prototyping (4)

- What do we do with the prototype once it has served the purposes already described?

  "In most projects, the first system built is barely usable. It may be too slow, too big, awkward in use or all three. There is no alternative but to start again, smarting but smarter, and build a redesigned version in which these problems are solved…When a new system concept or new technology is used, one has to build a system to throw away, for even the best planning is not so omniscient as to get it right first time."

Prototyping – Problems?

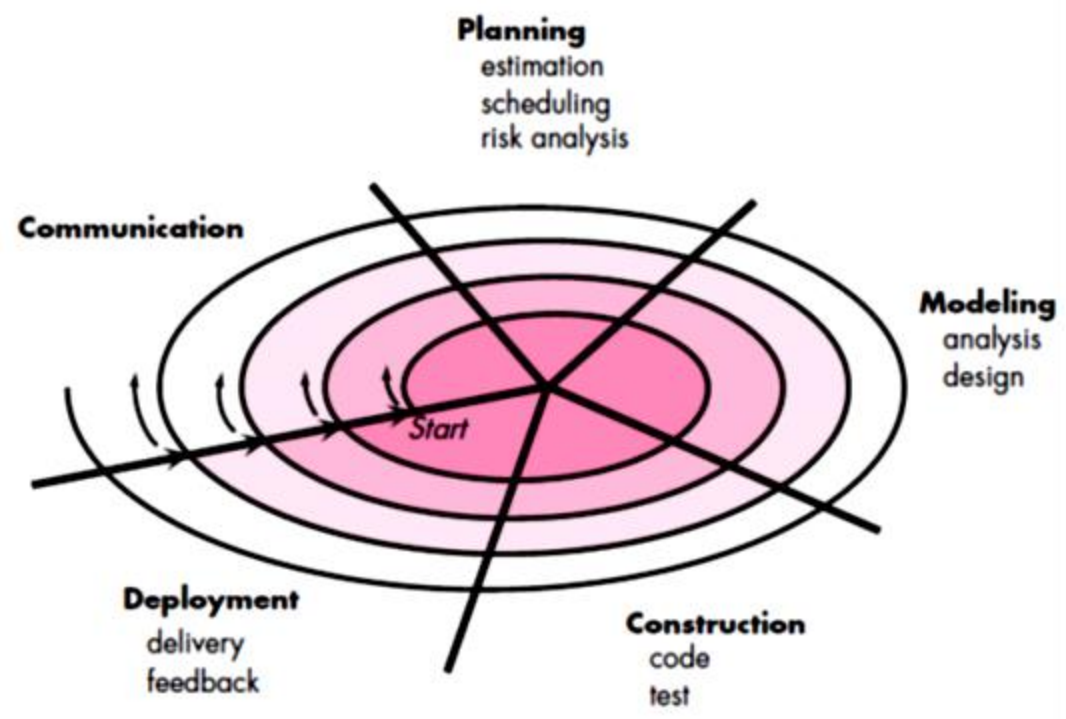
1) The customer wants the first prototype, unaware of it's shortfalls and compromises. The 1st prototype should be seen as "sacrificial" and be designed to be thrown away

2) Design decisions may be made to get a "quick and dirty" system running. The engineer may then loose sight of this fact and they may be retained in the final product

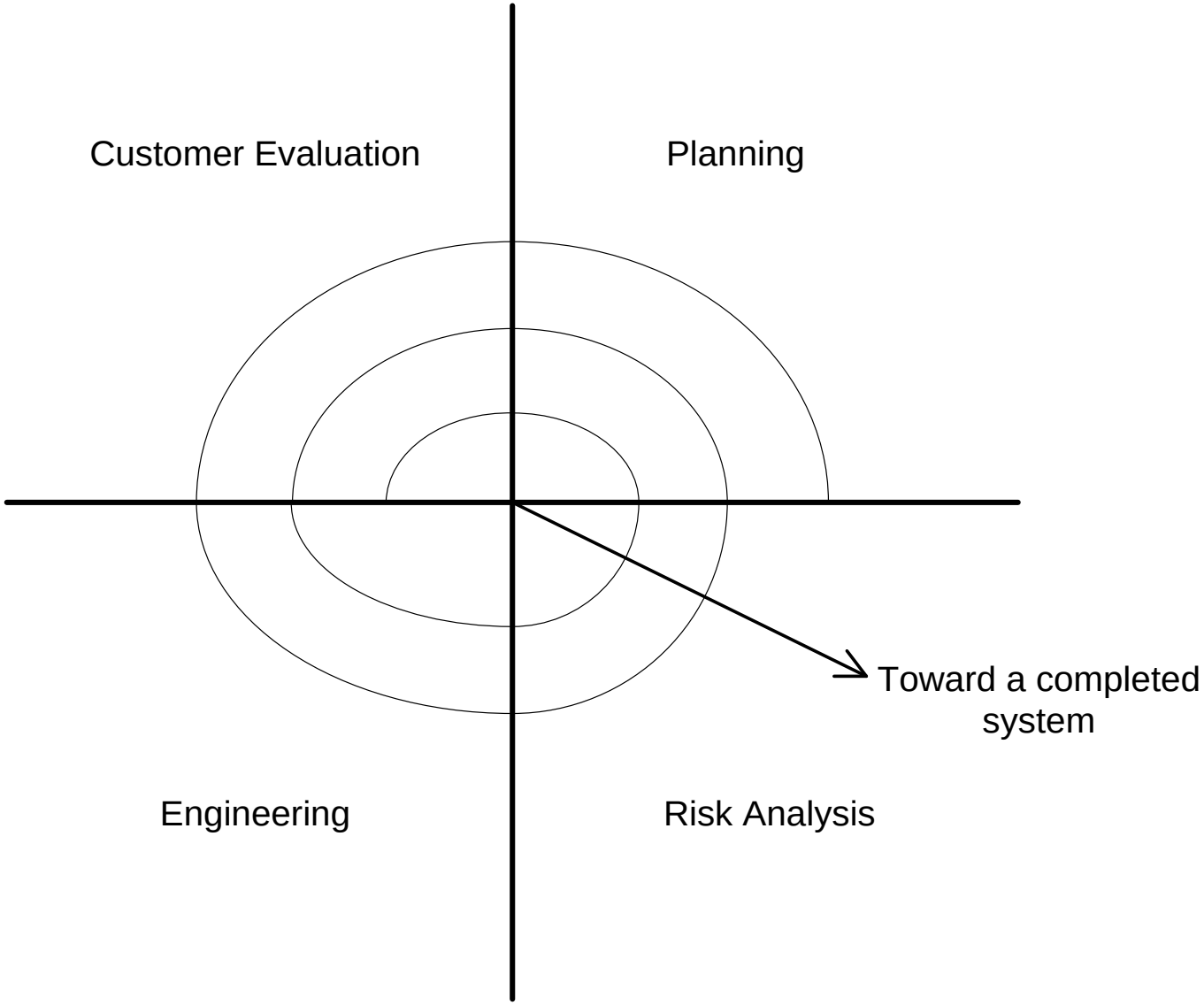
The Spiral Model (1)

- the spiral model is an evolutionary software process model that couples the iterative nature of prototyping with the controlled and systematic aspects of the waterfall model

- It provides the potential for rapid development of increasingly more complete versions of the software



The Spiral Model (2)



The Spiral Model (3)

- It is a modification of the prototyping loop with risk analysis added
- Planning
  - Refine the requirements spec
- Risk Analysis
  - What risks are we running in terms of the design decisions made?
  - Do we need to refine the design? If so we need to go round the loop again
- Engineering
  - 1st pass, a prototype is produced
  - Subsequent passes, the design is refined
- Customer Evaluation
  - 1st pass, the prototype is evaluated
  - Subsequent passes, the customer evaluates further developments

When to use Spiral Model

- When costs and risk evaluation is important
- For medium to high-risk projects
- Long-term project commitment unwise because of potential changes to economic priorities
- Users are unsure of their needs
- Requirements are complex
- New product line
- Significant changes are expected (research and exploration)

Advantages and Disadvantages of Spiral Model

Advantages

- High amount of risk analysis hence, avoidance of Risk is enhanced
- Good for large and mission-critical projects
- Strong approval and documentation control
- Additional Functionality can be added at a later date
- Software is produced early in the software life cycle

Disadvantages

- Can be a costly model to use
- Risk analysis requires highly specific expertise
- Project's success is highly dependent on the risk analysis phase
- Doesn't work well for smaller projects

Survey of Software Architectures

Goal: Control of Response Time

- Software is normally designed to accomplish a task in an efficient manner
- The primary concern about the design of software in embedded systems is to obtain the greatest amount of control over system response
  - Systems that require little control and poor response can be done with simple architectures
  - Rapid response systems will require more complex program design to be successful

Four Basic Architectures

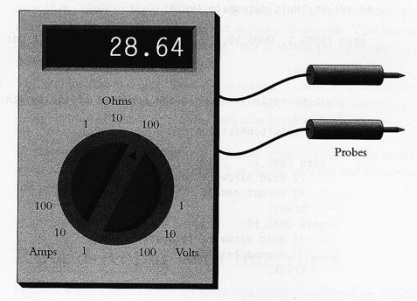
- We will cover four basic software architectures
  - Round-Robin (simplest)
  - Round-Robin with Interrupts
  - Function-Queue Scheduling
  - Real-time Operating Systems (most complex)

Round-Robin Architecture (1)

- This is a marvellously simple architecture
  - No interrupts
  - No shared data
  - No latency concerns
- Always an attractive potential architecture, as long as you can get away with it
- Consists of a main loop that checks each I/O device in turn and services them if needed
- Cannot suffer from shared data problems
- Latency is limited by the maximum duration of a loop cycle
- Attractive for simple environments

Example: Multimeter

- The multimeter example
  - very small number of I/O (switch, display, probes)
  - no particularly lengthy processing (even very simple microprocessors can check switch, take measurement and update display several times per second)
  - measurements can be taken at any time
  - display can be written to at any speed
  - small delays in switch position changes will go unnoticed



Round-Robin Problems

- If any device needs a response in less time than it takes the microprocessor to get around the main loop in the worst-case scenario, then the system won‘t work
  - if A and B take 5ms each and Z needs a response time of less than 7ms, then the processor won’t get to device Z quickly enough
  - this can be mitigate somewhat by doing (A,Z,B,Z) in a loop instead of (A,B,Z)
    - *Scalability of this solution is poor*
- Overall response time can easily become unacceptably poor
- This architecture is fragile. A single addition or change can ruin everything

Round-Robin with Interrupts

- This architecture gives a little bit more control over priorities
- Interrupt routines deal with the very urgent needs of the devices/hardware
- Interrupt routines set flags to indicate the interrupt happened
- Main while loop polls the status of the interrupt flags and does any follow-up processing required by a set flag
- Hardware interrupt signal cause the microprocessor to stop whatever it is doing in main function and execute the interrupt routine instead
- The processing of the interrupt routine has higher priority than task code in the main routine

Advantages - Round-Robin with Interrupts

- More control over priorities
  - device routines can be serviced in any order
  - processor interrupt priority settings can be used
- Interrupt routines get good response (low latency)
  - Main loop can be suspended
  - Interrupt routines are (must/should be) short
  - Interrupt code inherently has a higher priority than task code

Disadvantages - Round-Robin with Interrupts

- More complicated than Round-Robin
- Shared data problems
  - Care has to be taken to make sure the main code can safely be interrupted
- Latency and priority issues still exist
  - All task code executes at same priority
  - Task code could be moved to ISR but will increase execution time and increases response time of lower priority devices

Function-Queue Scheduling

- Improves the response time of higher priority tasks
- Interrupt routines add function-pointers to a queue for the main task to execute
- The main task continuously scans the queue and executes the corresponding function
- Allows placing function-pointers in the queue based on preferred priority scheme
  - Any task code that needs a quicker response can be executed earlier

Advantages: Function-Queue Scheduling

- Latency for high priority devices can be reduced compared to Round-Robin with Interrupts
  - (C,C,C,C,A,C,C,B) is simply a matter of the queuing algorithm
- In Round-Robin with Interrupts every loop may end up executing every follow-up task
  - Function-Queue Scheduling guarantees that at most a single follow-up task is executed per loop iteration

Disadvantages: Function-Queue Scheduling

- Latency for low priority tasks can increase
- Low priority tasks can actually starve
- Queuing algorithm may be complex/costly to run/code
- If a low priority follow-up task is very time consuming the latency for higher priority response times will suffer
  - Once you start processing this device you must wait for completion before any other task will get attention

Real-Time Operating Systems (RTOS)

- Interrupts signal the need for follow-up tasks
  - But, unlike Function-Queue Scheduling, this is handled by the Real-Time Operating System and not by the ISRs manipulating flags or a queue
- Instead of a loop deciding what to do next the RTOS decides
- One follow-up task can be suspended by the RTOS in favour of performing a higher priority task
  - Biggest difference compared to Function-Queue-Scheduling

Advantages - RTOS

- Suspension of tasks allows the worst case wait for the highest priority item to be zero
- Built-in scheduling mechanism yields a system with very stable response characteristics even when changes to the code occur
- Widely available for purchase

Disadvantages - RTOS

- Can be Costly
- RTOS are generally complicated and can consume a non-trivial amount of processor cycles

The Run Time Executive

- This is a mini operating system and will contain the following:
  - Interrupt Handler – The executive needs to respond to some interrupts e.g. the tick of the system clock
  - Real Time Clock – You need something which keeps track of time. A process should be able to ask the executive to ‘wake me up after 20ms’
  - Scheduler – This makes use of the real time clock to periodically get control and decide which process should run next

Architecture Selection

- Select the simplest architecture that will meet your response requirements
- If your response requirements might necessitate using a real-time operating system then that should probably be your choice
  - Things rarely get smaller/simpler and its a lot easier to start on a more complicated architecture than to migrate to it later when things grew too hairy
- If it makes sense create hybrids

Software Testing

Software Testing

- This is a integral part of the development, and should be designed in to the development process
- Testing Strategies are usually based around the following points:
  - Test individual modules before combining them into the complete system
  - Use a range of testing techniques
  - Both the developer and someone independent will perform the testing
  - Testing and debugging are separate activities. Document the testing procedure and the result well so they can easily be repeated
  - Testing will involve data which will force the system into error states. Can your system cope with this?

Testing Fundamentals

- Testability: is simply how easily a computer program can be tested
- Test Characteristics: And what about the tests themselves?
  - A good test has a high probability of finding an error
  - A good test is not redundant
  - A good test should be best of breed
  - A good test should be neither too simple nor too complex

White Box

- Inspect the inner workings of the component and test these against the specification
- In software terms this relates to looking at a path through the program, comparing it to the path required to solve the problem and then repairing any discrepancies

White Box Testing

- Using white box testing methods, the software engineer can derive test cases that:
  1. Guarantee that all independent paths within a module have been exercised at least once
  2. Exercise all logical decisions i.e. both true and false
  3. Execute all loops at their boundaries and within their operational bounds
  4. Exercise internal data structures to ensure their validity

Black Box Testing

- Treat the component as a device which accepts inputs and produces outputs
- Apply inputs and note the correspondence between the outputs and those required

Black Box Testing

- Black Box Testing attempts to find errors in the following categories:
  1. Incorrect or missing functions
  2. Interface errors
  3. Errors in data structures or external database access
  4. Performance errors
  5. Initialisation and termination errors

Grey Box Testing

- Grey box testing is the combination of black box and white box testing
- Intention of this testing is to find out defects related to bad design or bad implementation of the system
- The test engineer is equipped with the knowledge of system and designs test cases or test data based on system knowledge

Integration Testing

- Isn't it legitimate to assume that once all the modules have been individually tested that they will all work together?
- The problem is putting them together, or a better word for this is *interfacing*
- One interface/module can have an adverse effect on another

Types of Strategies - Bottom Up

- Each low level component is tested in a test harness before integration into the system
- Fit two components together and test them
- Progressively integrating the components together until the entire system is tested
- Problems with this strategy:
  - We only test the top module last of all. Therefore any problems with this will not be apparent until the end of the project
- Bottom Up is very popular for hardware based work, e.g. mouse organ

Types of Strategies - Top Down

- This is the opposite of bottom up
- The top level interface is written first and tested
- Once the overall design has been decided on we place empty function stubs in the code to take the place of the finished routines. As progress is made more and more of these stubs will be replaced with working code.
- Problems with this strategy:
  - The function stubs can be hard to write
  - The order they are written can be critical
  - Testing of some functions can only be achieved when other critical functions have been written and tested
- Top Down is very popular for entirely software based projects

Types of Strategies - Big Bang

- Really a combination of ‘Top Down’ and ‘Bottom Up’
- Both ends of the development integrate towards a defined middle layer
- The user interface developers make use of the functions stubs approach while the hardware developers use bottom up test harnesses
- When both ends have been proven they are linked across the middle layer and the system is then tested as a whole.
  - Applicable to mouse organ

Verification & Validation

- You should worry about two things when testing the system:
  - Verification – Are we building the product right?
  - Validation – Are we building the right product?
- Verification is to ensure you are using the right production process
- Validation is to ensure that the software has been built in line with the customers requirements

Further Software Testing

Testing Embedded Software

- Generally the traits that separate embedded software from applications software are:
  - Embedded software must run reliably without crashing for long periods of time
  - Embedded software is often used in applications in which human lives are at stake
  - Embedded systems are often so cost-sensitive that the software has little or no margin for inefficiencies of any kind
  - Embedded software must often compensate for problems with embedded hardware
  - Real-world events are usually asynchronous and nondeterministic, making simulation tests difficult and unreliable
  - Your company can be sued if your code fails!!!

Testing Embedded Software

- Because of these differences, testing of embedded software differs from applications testing in four major ways:
  1. Because real-time and concurrency are hard to get right, a lot of testing focuses on real-time behaviour
  2. Because most embedded systems are resource constrained real-time systems, more performance and capacity testing is required
  3. You can use some real-time trace tools to measure how well the tests are covering the code
  4. You'll probably test to a higher level of reliability than if you were testing applications software

Testing Embedded Software

- In every real-time system, certain combinations of events cause the greatest delay from an event trigger to the event response
  - Critical sequences
- The test suite should be capable of generating all critical sequences and measuring the associated response time
- A time critical failure is a failure to meet an important timing deadline
  - Sometimes known as hard real-time failure
- A time sensitive failure is really poor performance or high latency
  - Sometimes known as soft real-time failure

Testing Embedded Software

- Another category of failures is created when the system is forced to run at or near to full capacity
- You may never see a `malloc()` error when the system is running at half load, but at three quarters it may occur
- Many RTOS's use fixed size queues to track waiting tasks and buffer I/O.
  - What happens if the system receives an usually high number of asynchronous while it is heavily loaded?
  - Do the queues fill up?
  - Is the system still able to meet deadlines?

Measuring Test Coverage

- Even if you use both Black Box and White Box testing methodologies it is unlikely you will test all of the code.

- So how do we measure test coverage?
  - Software Instrumentation.
  - Hardware Instrumentation.

Software Instrumentation

- A software only measurement method based upon execution triggering
- Coverage can be measured by inserting trace calls at the beginning of each 'basic block' of sequential statements
- In this context a basic block is a set of statements with a single entry point at the top and one or more exits at the bottom
- Each control structure, such as a `goto`, `return`, or `decision`, marks the end of a basic block.
- The implication is that after a block is entered every statement in the block is executed
- By placing a simple trace statement, such as a `printf()`, at the beginning of every basic block, you can trace when the block is executed. By implication every all of the statements in the block are also executed!

Software Instrumentation

- Problems with this technique are:
  - `printf()` statements slow the systems down considerably, hence not low intrusion testing
  - Small deeply embedded systems may not have a convenient way of displaying the output
- Alternatives:
  - If using a RTOS, it may have a low intrusion logging service which your trace code can call at the entry point of every basic block
  - Even less intrusive is a method where a simple memory write is used instead of `printf()`. The logging function 'marks' a unique spot in memory when a block is called. Requires external software to check results

Hardware Instrumentation

- Three general purpose tools exist in this category:
  - Emulation Memory
  - Logic Analyser
  - Software Performance Analysers

Emulation Memory

- Part of an ICE – in circuit emulator.
- Some vendors include a coverage bit in the attribute bits in their emulation memory.
- This bit is set when the memory location is accessed. These can then be used later to work out percentage of coverage.

Logic Analyser

- Can record memory access activity in real time.
- Difficult to convert its trace data into coverage data.
- Could use statistical sampling, but a PC connected to the logic analyser is required.

Software Performance Analysers

- A hardware collection tool that performs low intrusion collection without requiring a logic analyser.
- Many ICE vendors manufacture hardware based tools for analysing test coverage and software performance.
- By using the information from the linkers load map these tools can display coverage information on a function or module basis.